EPHEC
ÉCOLE SUPÉRIEURE
ENSEIGNEMENT POUR ADULTES

Année académique **2026-2027**

Support de cours - Programmation orientée objet - Java

Module PKG

Packages et visibilité

Qui a le droit de voir quoi

EPHEC — Bachelier en informatique, niveau 2

Les deux modules précédents ont montré comment des classes partagent du code et collaborent. Celui-ci pose une question laissée de côté jusqu'ici : qui a le droit de voir quoi.

Vous avez écrit jusqu'ici `private` devant les champs et `public` devant le reste, souvent par habitude. Ce module donne les raisons de ce choix, et y ajoute un niveau que la plupart des programmeurs emploient sans le savoir. La question n'est pas théorique : elle se pose à chaque classe de la quatrième question de l'examen, où il faut décider ce que la classe mère montre à ses filles, et ce qu'elle cache au reste du programme.

> **À la fin de ce module, vous saurez**
>
> - ranger une classe dans un package, et l'utiliser depuis un autre grâce à `import` ;
> - prévoir ce qu'un membre laisse voir selon son modificateur d'accès, y compris quand on n'en écrit aucun ;
> - choisir entre un champ public et un accesseur, en sachant ce que chacun engage ;
> - réduire la surface publique d'une classe au strict nécessaire ;
> - découper un programme en packages, et déplacer une classe sans rien casser ;
> - placer `protected` parmi les quatre niveaux d'accès.

PKG §1 — Un package, un dossier

Ranger une classe

```
// src/main/java/be/ephec/biblio/modele/Livre.java

package be.ephec.biblio.modele;

public class Livre {
    // ...
}
```

La ligne `package` ouvre toujours le fichier. Vous avez rencontré la règle au module `POO`, avec la classe `Personne` du package `progoo`. Elle se précise ici : chaque segment du nom correspond à un dossier, et un nom à plusieurs segments correspond à autant de dossiers imbriqués.

Le nom va du général au précis : l'organisation qui produit le code, le projet dans le contexte duquel il est produit, puis le module. L'usage est de commencer par le nom de domaine de l'organisation, à l'envers : `be.ephec` pour l'EPHEC. Deux équipes qui écrivent chacune une classe `Livre` ne se gênent alors jamais. Les exercices suivent la même logique, avec `be.ephec.java.pkg.e01` pour le premier exercice de ce module.

Un nom de package s'écrit entièrement en minuscules. IntelliJ crée les dossiers quand vous créez un package, et signale tout écart entre la ligne `package` d'un fichier et le dossier où il se trouve.

Le nom complet d'une classe

```
package be.ephec.biblio.service;

import be.ephec.biblio.modele.Livre;   // nom complet
import java.time.LocalDate;

public class Emprunt {
    private Livre livre;                // nom court
    private LocalDate dateRetour;
}
```

Le nom complet d'une classe commence par son package : celui de `Livre` est `be.ephec.biblio.modele.Livre`. L'instruction `import` permet ensuite d'écrire le nom court. Un `import` ne copie rien et ne charge rien : c'est un raccourci d'écriture, qui dit au compilateur où chercher quand il rencontre le nom court.

- **Une classe du même package ne nécessite pas d'`import`.**
- **Les classes de `java.lang` non plus**, comme `String`, `Math` ou `System` : le compilateur les importe d'office, ce qui explique que vous ne les ayez jamais importées.
- `import java.util.*;` importe toutes les classes du package `java.util`, mais pas celles de ses sous-packages.
- **Quand deux classes portent le même nom court**, `java.util.Date` et `java.sql.Date` par exemple, il faut écrire le nom complet d'au moins l'une des deux.

Un sous-package est un package à part

```
be.ephec.biblio              Main
be.ephec.biblio.modele       Livre, Auteur
be.ephec.biblio.service      Emprunt, Catalogue

// trois packages, sans lien de parente
```

Les noms de packages forment une arborescence, mais pas les droits. L'idée fausse est tenace : la hiérarchie existe dans les noms et dans les dossiers, mais pour le compilateur, `be.ephec.biblio` et `be.ephec.biblio.modele` sont deux packages aussi étrangers l'un à l'autre que `java.util` et `java.time`. Un package n'a aucun accès privilégié à ses sous-packages, ni l'inverse, et deux sous-packages voisins sont deux packages distincts.

L'arborescence est donc un rangement, pas une famille. Elle aide le lecteur à s'y retrouver, et c'est tout. Gardez-le en tête pour `PKG.2`, dont les deux classes vivent dans `e02.coffre` et `e02.controle`.

Deux dossiers, un seul package

```
src/main/java/be/ephec/biblio/modele/Livre.java
src/test/java/be/ephec/biblio/modele/LivreTest.java

// les deux fichiers commencent par la meme ligne :
package be.ephec.biblio.modele;
```

Maven range le code dans `src/main/java` et les classes de test dans `src/test/java`, pour ne pas livrer les seconds avec le premier. Mais une classe de test déclare le même package que la classe qu'elle teste, et Maven les compile dans ce même package. Le package est une notion du langage, le dossier racine une convention de Maven.

Une classe de test et la classe qu'elle teste sont donc voisines au sens strict, même si l'arborescence d'IntelliJ les affiche dans deux branches différentes. Toutes les classes de test du recueil sont construites ainsi, et la section suivante dit pourquoi ce voisinage compte.

PKG §2 — Le niveau sans mot-clé

Qui peut atteindre le membre d'une classe

```
package be.ephec.biblio.modele;

public class Livre {

    private String isbn;               // private : la classe Livre
    String cote() { ... }              // defaut : le package modele
    public String titre() { ... }      // public : tout le programme
}
```

Un mot de vocabulaire d'abord. Un *membre* d'une classe, c'est l'un de ses champs ou l'une de ses méthodes. Les mots `public`, `private` et les autres s'appellent des *modificateurs d'accès*. Ne les confondez pas avec les accesseurs, qui sont des méthodes comme `getTitre` (voir `PKG §3`).

- **Le niveau `private`** : le code de la classe seul.
- **Le niveau `public`** : tout le programme.
- **Sans modificateur** : le package, et nulle part ailleurs.

Les modificateurs s'appliquent aussi aux constructeurs et à la classe elle-même. Il en existe un quatrième, `protected`, que vous avez rencontré au module `ABS` : il attendra `PKG §6`, parce qu'il ne se comprend qu'avec l'héritage.

Retenez les mots exacts de la définition de `private` : *le code de la classe*, et non *l'objet*. `PKG.1` vous demande ce que cela implique quand deux objets de la même classe se rencontrent. Essayez avant de conclure.

Le niveau qu'on choisit sans le savoir

```
package be.ephec.biblio.modele;

class Fiche {                  // invisible hors du package
    String cote;               // idem
    Fiche(String cote) {       // idem
        this.cote = cote;
    }
}
```

Le niveau sans mot-clé s'obtient en n'écrivant rien. Il s'appelle *visibilité de package*, ou visibilité par défaut, et vaut pour les membres, les constructeurs et la classe elle-même. Le module `POO` l'avait signalé en passant : la classe `Personne`, déclarée sans modificateur, n'était visible que dans son package.

Son danger est que rien ne signale qu'on l'a choisi. Tant que toutes les classes vivent dans le même package, ce niveau ne se distingue pas de `public` : tout compile, partout. Il se comporte comme `private` pour les autres packages, et on ne le découvre que le jour où le programme est découpé.

Il a pourtant un usage délibéré, qui découle de `PKG §1` : une classe de test vit dans le même package que la classe testée. Un membre laissé à ce niveau lui est accessible, sans être ouvert au reste du programme.

> **Démonstration** - `Demo_PKG` : déménager une classe, et voir ce qui casse
>
> La démonstration part d'un programme qui fonctionne, entièrement rangé dans un seul package, et le découpe en trois avec l'outil de refactoring. La compilation se brise aussitôt. Chaque erreur désigne un endroit où le code s'appuyait sur une proximité qui vient de disparaître, et le compilateur pose à chaque fois une question de conception.

> **Exercices** - PKG.1 Ce que voit une classe du même package
>
> L'exercice est une expérience : six lignes commentées, une par membre. Décommentez-les une à une et notez celles que le compilateur refuse. La liste attendue par le test se déduit de ce que vous avez observé, pas l'inverse. La première question de fin d'exercice porte sur le périmètre exact de `private`, et la réponse surprend souvent.

PKG §3 — Champ public ou accesseur

Ce qu'un champ public promet

```
public class Livre {

    public int exemplaires;
}
```

```
// ailleurs, sans que Livre le sache
livre.exemplaires = -3;
```

Un champ public donne deux droits à la fois, lire et écrire, à tout le monde et pour toujours. La classe perd la main sur sa propre donnée : rien ne l'empêche de valoir moins trois, et rien ne la prévient quand c'est le cas.

Le coût le plus lourd vient à long terme, parce que d'autres classes dépendent du champ. Le jour où vous voudrez le renommer, changer son type ou le remplacer par un calcul, toutes les classes qui l'emploient devront changer avec lui. Dans une équipe, ces classes ne sont pas toutes les vôtres.

Ce que l'accesseur garde

```
public class Livre {

    private int exemplaires;
    private int empruntes;

    public int getDisponibles() {       // hier : un champ
        return exemplaires - empruntes;
    }
}
```

Un accesseur public ne donne que la lecture. La classe peut calculer la valeur, la formater ou la refuser, et son corps change sans qu'aucun appelant ne le sache. La méthode `getDisponibles` retournait hier un champ ; elle fait aujourd'hui un calcul, et aucun appelant n'a eu à changer.

C'est la réponse à une question que beaucoup se posent depuis le module POO : à quoi bon un getter qui se contente de retourner un champ ? Le jour où il fait autre chose, personne ne doit le savoir, et c'est précisément ce qu'il permet. Le getter et le champ public ne sont donc pas équivalents, même quand leur code se ressemble : l'un expose une donnée, l'autre un comportement, celui de la donner.

L'autre moitié de la règle CC-OBJ.2 compte autant : un accesseur n'est pas une obligation. Écrire un getter et un setter pour chaque champ revient à rendre les champs publics avec plus de lignes. Chaque accesseur est une décision.

> **Exercices** - `PKG.2` Ce que voit une classe d'un autre package
>
> La première partie reprend l'expérience de `PKG.1`, avec une seule différence : le package. Notez lequel des six membres change de camp, et rapprochez-le de `PKG §2`. La seconde partie impose une contrainte qui est toute la consigne : le champ reste tel qu'il est. Le passer en `public` ferait compiler l'exercice en dix secondes, et l'un des deux tests est écrit pour le refuser.

PKG §4 — La surface publique

Chaque membre public est une promesse

On appelle *surface publique* l'ensemble de ce qu'une classe montre aux autres packages. C'est la partie de la classe sur laquelle d'autres s'appuient, et donc la seule qu'on ne peut plus modifier sans prévenir.

Une asymétrie décide de tout. Une méthode privée qu'on rend publique ne casse rien, puisque personne ne l'utilisait. Une méthode publique qu'on rend privée casse tous ceux qui l'appelaient, et on ne sait pas toujours qui ils sont. D'où la règle : partir du plus fermé, et n'ouvrir que sur un besoin réel. Ouvrir par défaut, c'est s'interdire de changer d'avis ; fermer par défaut laisse toutes les portes ouvertes.

Choisir, membre par membre

```
public class Catalogue {

    private Livre[] livres = new Livre[100];

    public void ajouter(Livre livre) { ... }
    public Livre chercher(String isbn) { ... }

    boolean estCoherent() { ... }       // pour les tests
    private void agrandir() { ... }
}
```

1. **Le niveau `private` d'abord** : c'est le cas le plus fréquent.
2. **La visibilité de package**, si une voisine en a besoin (par exemple pour les tests).
3. **Le niveau `public`** pour ce que les autres appellent.

Lisez la classe en partant de ce qu'un utilisateur du catalogue doit pouvoir faire : ajouter un livre, en chercher un. Ce sont les deux seules méthodes publiques. Le tableau et la façon de l'agrandir sont des détails de fabrication, et restent privés. La méthode `estCoherent` est un cas intermédiaire : elle vérifie que le catalogue n'est pas corrompu, ce qui intéresse les tests mais pas les utilisateurs. Une voisine, ici, c'est d'abord la classe de test : la visibilité de package la met à sa portée, et la cache au reste du programme.

Le choix se fait donc membre par membre, et une question suffit : qui a réellement besoin de ceci ?

Une seule classe publique par fichier

```
// fichier Catalogue.java

public class Catalogue { ... }

class Rayon { ... }             // permis : visible dans le package

public class Etagere { ... }    // erreur de compilation
```

La surface publique se mesure aussi en classes. Une classe est publique ou de package, rien d'autre : `private` et `protected` sont interdits sur une classe écrite directement dans son fichier. Une classe sans modificateur est un détail d'implémentation de son package, et les autres packages l'ignorent.

Un fichier contient au plus une classe publique, et porte son nom. Ce n'est pas une convention mais une règle du compilateur : il exige qu'une classe publique `Etagere` vive dans un fichier `Etagere.java`, et il le dit dans son message. Rien n'interdit de placer plusieurs classes non publiques dans un même fichier, mais l'usage est d'en écrire une par fichier, et IntelliJ s'y range.

> **Exercices** - `PKG.3` Réduire la surface publique d'une classe
>
> La classe fournie fonctionne, et tout y est public. Le programme fourni définit ce dont un utilisateur a réellement besoin : il n'appelle que trois opérations. Le travail consiste à fermer tout le reste sans que ce programme cesse de compiler. Comptez les promesses que faisait la version d'origine, et celles que fait la vôtre : la différence mesure le travail accompli. Voir `CC-CLS.2`.

PKG §5 — Découper en packages

Ranger par responsabilité

```
be.ephec.biblio.modele          Livre, Auteur
be.ephec.biblio.service         Emprunt, Catalogue
be.ephec.biblio.application     Main
```

Un package regroupe des classes qui changent pour les mêmes raisons. C'est le critère de la règle `CC-CLS.1`, appliqué à un groupe de classes. Un découpage courant compte trois couches :

Couche	Ce qu'elle contient	Elle change quand
le modèle	les données du domaine	le domaine change
les services	les règles de gestion	une règle change
l'application	le lancement, l'affichage	la façon de lancer ou d'afficher change

Le modèle est la couche la plus précieuse, parce qu'elle ne dépend d'aucune autre : la même classe `Livre` peut servir demain dans une application graphique sans qu'une ligne y change. Une quatrième couche, la persistance (ou *data*), écrit les données dans une base ou un fichier ; elle viendra avec le module `FIC`.

Les imports dessinent les dépendances

```
application  ->  service  ->  modele

// les classes du modele n'importent
// ni les services, ni l'application
```

Les `import` d'une classe disent de quoi elle dépend. Tant que tout vit dans un seul package, les dépendances existent mais ne se voient pas ; le découpage les rend lisibles.

Lisez la flèche comme « connaît ». L'application connaît les services, qui connaissent le modèle. Le modèle ne connaît personne, et c'est ce qui le rend réutilisable. Dans un programme bien découpé, les dépendances vont toutes dans le même sens, et une dépendance qui remonte signale une responsabilité rangée dans la mauvaise couche.

Déplacer une classe avec l'outil

Pour déplacer une classe à l'aide d'IntelliJ : clic droit sur elle dans l'arborescence, puis Refactor, puis Move, ou directement `F6`. IntelliJ demande le package de destination, déplace le fichier, réécrit sa ligne `package` et met à jour tous les `import` du projet (voir `IDE.21`). Sur un clavier de Mac, la touche `F6` demande parfois d'appuyer en même temps sur `fn`.

C'est le même principe que les refactorings du module `ABS` : l'outil connaît toutes les références, vous non. Un déplacement à la main garde une ligne `package` fausse et oublie toujours un `import`. Un copier-coller laisse deux classes du même nom dans deux packages, ce qui compile parfaitement et ne fait jamais ce qu'on croit.

> **Exercices** - `PKG.4` Découper un programme en packages
>
> Quatre classes, un seul package, trois packages à créer. Faites-le avec `F6` : le dernier test vérifie qu'aucune classe ne subsiste à l'ancienne place, ce qui vous protège d'un copier-coller. Une fois le découpage fait, regardez les `import` de chaque classe et dessinez les flèches. Puis relisez `PKG §2` : certaines classes s'appuyaient sur la visibilité de package de leurs voisines. Le sont-elles encore ? Voir `CC-CLS.1`.

PKG §6 — Le quatrième niveau : `protected`

Le rappel du module `ABS`

```
package be.ephec.biblio.modele;

public abstract class Document {
    private final String cote;
    protected int pages;

    protected Document(String cote) {
        this.cote = cote;
    }
}
```

Vous avez employé `protected` au module `ABS`, pour un champ que les filles devaient lire, et pour les méthodes que la mère appelle et que les filles écrivent. Ce module lui donne sa place exacte parmi les quatre niveaux.

- **Le niveau `protected` ouvre un membre aux sous-classes.**
- **Il l'ouvre aussi au package**, ce qu'on oublie souvent : un membre `protected` est visible de tout le package, sous-classe ou non. Il est donc plus ouvert que la visibilité de package, jamais moins.
- **Son effet propre ne se voit que pour une sous-classe d'un autre package.** Dans le même package, `protected` ne change rien. C'est exactement la situation de `PKG.5`.

Les quatre niveaux

Niveau	Classe	Package	Sous-classe ailleurs	Reste
`private`	oui	non	non	non
(rien)	oui	oui	non	non
`protected`	oui	oui	oui	non
`public`	oui	oui	oui	oui

Le tableau se lit de haut en bas comme une échelle : chaque niveau ouvre tout ce qu'ouvre le précédent, plus une colonne. C'est ce qui permet de le reconstruire sans l'apprendre.

La colonne « sous-classe ailleurs » désigne une sous-classe rangée dans un autre package. Une sous-classe du même package est d'abord une voisine : elle relève de la deuxième colonne. Dans une interface, la question ne se pose presque pas : comme vu au module `ITF`, ses méthodes sans corps sont publiques sans qu'on l'écrive.

Choisir `protected`

```
public abstract class Document {

    private int pages;                    // la mere

    protected int getPages() {            // les filles
        return pages;
    }

    protected abstract String format();   // idem
}
```

Un membre `protected` est une promesse faite à toutes les sous-classes, présentes et futures. La règle de `PKG §4` s'applique donc sans changement : une sous-classe peut être écrite par quelqu'un d'autre, dans un autre package, des mois plus tard, et tout ce qui lui est ouvert ne peut plus changer sans la casser.

Un champ `protected` laisse les filles lire et écrire, comme le faisait le champ public de `PKG §3`, devant un public plus petit. Un champ privé et un accesseur `protected` laissent la mère maîtresse de la valeur. Le niveau `protected` trouve son meilleur usage dans les méthodes que la mère appelle et que les filles écrivent, comme au module `ABS`.

Ce n'est pas une interdiction : `PKG.5` vous fait écrire les deux versions, et la question de fin vous demande laquelle vous préférez, et pourquoi.

> **Exercices** - `PKG.5 protected`, ou ce que l'héritage ouvre
>
> L'exercice place deux classes dans le même package, autour d'une classe mère rangée ailleurs. L'une en hérite, l'autre s'en sert seulement : les deux sont voisines et n'ont pourtant pas les mêmes droits. Le cœur de l'exercice est le tableau d'expériences : trois champs, deux classes, six essais. Notez le message exact du compilateur à chaque refus, puis écrivez en une phrase ce que `protected` ouvre. `ABS.4` vous demandait quelle visibilité donner aux accesseurs intermédiaires de la hiérarchie : vous avez maintenant tout ce qu'il faut pour répondre.

Ce qu'il faut retenir

- Un package est un dossier ; un sous-package est un package à part, sans droits sur son parent.
- Sans modificateur, un membre est visible du package, et seulement du package.
- Un champ public expose une donnée ; un accesseur, un comportement. Et chaque accesseur est une décision.
- La surface publique se réduit au strict nécessaire : partir du plus fermé, n'ouvrir que sur un besoin réel.
- Les packages se découpent par responsabilité, et les dépendances vont toutes dans le même sens.
- `protected` ouvre aux sous-classes, et aussi au package.

La leçon qui dépasse ce module tient en une asymétrie : on peut toujours ouvrir plus tard, on ne peut presque jamais refermer.